A reliable ledger.
A helpful assistant.

What we learned from Spiich, Midday and Actual—and how those lessons shape a focused, explainable accounting product.

You describe the work

“Paid $2,400 rent from checking.”

→

AI interprets it

Suggest accounts; ask when facts are missing.

→

Application controls it

Review, validate, and permit the operation.

→

Database protects it

Balance, exact values and preserved history.

Spiich / useful work

Connect ordinary language to a business record and a permitted action. Its connection to the assignment is our inference.

Midday / factual tools

Use authored financial queries and supporting records. Its founders announced a Ramp transition and product wind-down.

Actual / visible rules

Make repeated category decisions inspectable and editable. Its SQLite budget model differs from our PostgreSQL journal.

Recommendation: complete a Gemini baseline on the existing ledger, simplify the main tasks, and let measured failures decide whether another component earns its place.

Planning snapshot before the Gemini/interface implementation. See CHECKPOINT.md for current status. Examples are illustrative; sources are on page 8.

Built, proposed, and still to verify.

Code existing and behavior being proved in the real environment are different claims.

BUILT / IMPLEMENTED

PROPOSED / NEXT CHOICE

VERIFY / EVIDENCE NEEDED

AREA	BUILT TODAY	PROPOSED NEXT	STILL TO VERIFY
Ledger + reports	Exact PostgreSQL journal, balanced posting, reversals and four SQL reports.	Preserve the engine; explain results clearly.	Relevant regressions after changes.
AI entry + chat	OpenAI adapter, proposals, clarification and four query families.	Gemini adapter; stronger evaluation.	No key configured; no live model tests.
Categorization	Text retrieval of history and confirmed corrections.	Explicit rules; optional semantic retrieval.	Whether suggestions improve.
Background work	Recurring drafts, statistical alerts, durable jobs and retries.	Retain bounded processing.	Final environment behavior.
Interface	All assignment workflows and Daybook redesign.	Simpler record / review / understand flow.	Revised flows; physical phone.
Docker + delivery	Compose files, config check, archive and documentation.	Clean startup and persistence checks.	Actual Docker build/start pending.

29 **Database / app tests**
Real PostgreSQL

8 **Provider-contract tests**
Controlled responses; not live accuracy

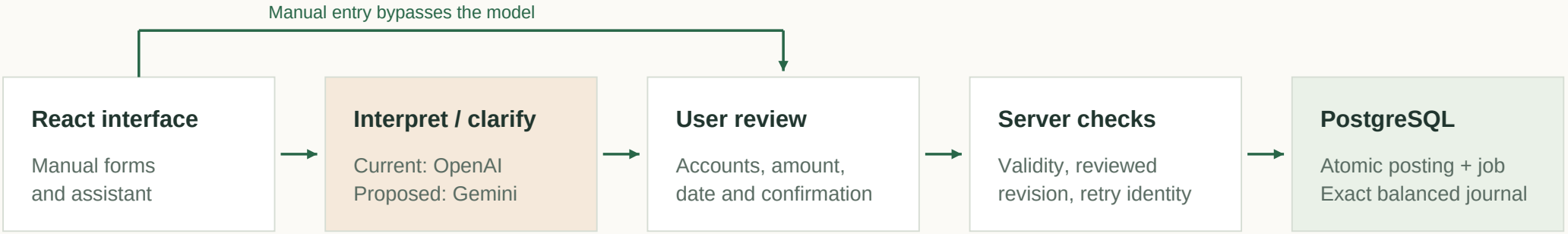
8 **Browser workflows**
Production bundle; not physical devices

Saved historical baseline. Report creation did not rerun these suites. Authentication, million-user capacity, failover and disaster recovery are not production claims.

Source: CHECKPOINT.md / docs/VERIFICATION.md

Keep authority where it can be enforced.

AI proposes an interpretation. The user confirms it. Server code and PostgreSQL enforce posting.



Approved report functions

SQL reads posted history; application returns values and evidence.

Read-only query path

Background modules share the API process today.
These boxes are not deployed microservices.

Recurring scheduler

An independent timer creates drafts for review.

Anomaly processing

Stored jobs run after commit and create review alerts.

Protect the money

Balance is checked at posting in base currency. Drafts can be incomplete. Corrections use reversals.

Protect the meaning

A balanced invoice payment can still double-count revenue. Missing facts require clarification.

Protect the operation

Retries must not double-post; stale reviews must fail. Job delivery is at least once.

Production authorization additionally needs authenticated users and enforced organization membership.

Recording money and answering questions.

A write changes the books. A read explains the books. Keep those paths easy to distinguish.

WRITE PATH / LIVE AI PENDING

“Paid $2,400 rent from checking on March 1.”

1 Interpret. Identify accounts, amount and date; ask for missing context.
2 Validate. Check the proposed values and accounting rules.
3 Review. The user sees and confirms the exact entry.
4 Post. Revalidate the revision; commit lines and analysis job together.
5 Explain. Show the saved entry and its effect.

Rent expense	Debit $2,400.00
Checking	Credit $2,400.00

USD example: expenses rise by $2,400; checking falls by $2,400. Posted history remains unchanged.

READ PATH / LIVE AI PENDING

“What were my top expenses last month?”

1 Resolve the period. Establish the dates.
2 Choose an approved function. Expense summary, not generated SQL.
3 Query the posted journal. Apply organization, date and account filters.
4 Return facts and evidence. Show dates, currency and supporting entries.

> **Anomaly example:** previous rent is $2,400; a new payment is $4,800. A statistical rule can flag the change. Historical records suffice.

Current Daybook composes numeric answers in application code. Alerts are review hints, not proof of fraud.

Good failure demo: omit a payment account or exchange rate and show a clarification.

These tools solve different problems.

Start with a complete baseline. Add a component when a requirement or repeatable failure explains it.

PostgreSQL

BUILT / KEEP

Authoritative records, exact queries and constraints.

Gemini

PROPOSED BASELINE

Interpret sentences and choose approved tools. Needs adapter and live checks.

Mastra

CONDITIONAL

Coordinate models, tools, workflows and traces if our code needs it.

Jev

CONDITIONAL

A narrow category decision or check; measure errors, cost and latency.

pgvector

EXPERIMENT

Find similar history or corrections. SQL still calculates totals.

Managed file search

IF DOCUMENTS ADDED

Index and retrieve uploaded material. Document ingestion is extra scope.

Wren

BROADER ANALYTICS

Business definitions and query infrastructure beyond the four query families.

n8n / load balancer

DIFFERENT LAYERS

n8n connects workflow steps. A load balancer spreads server requests.

Midday: targeted retrieval

Embeddings select relevant tool descriptions. Current receipt matching uses names, dates, amounts and feedback.

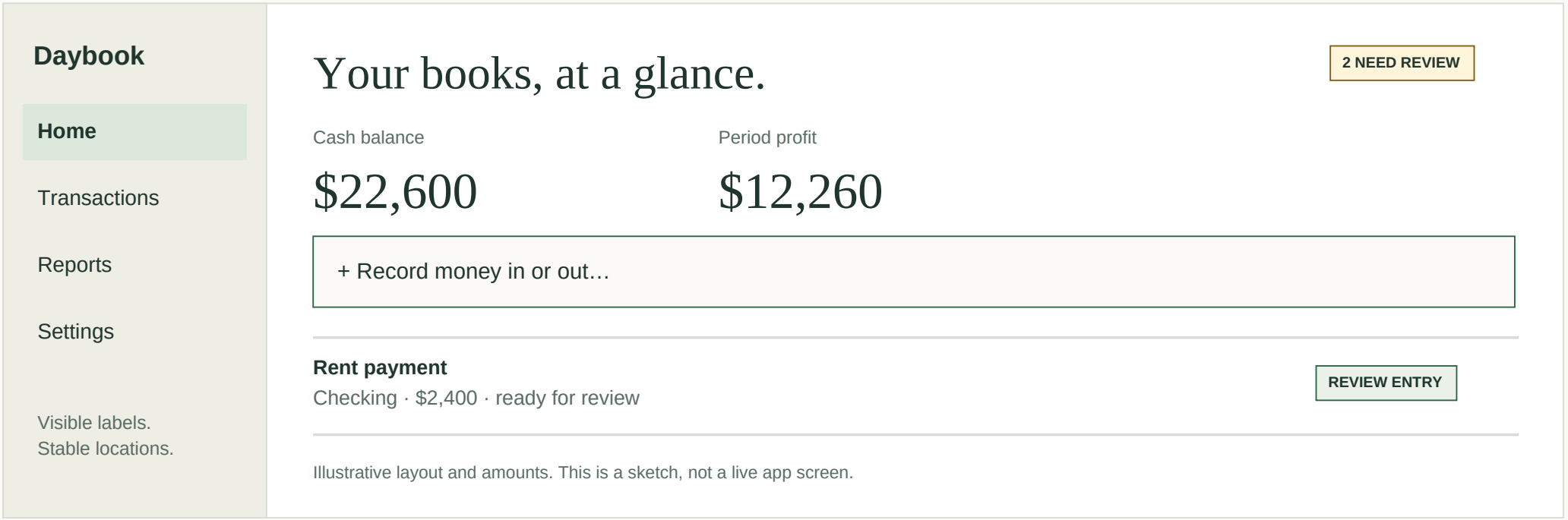
Actual: inspectable rules

A confirmed payee-to-category rule can be repeatable. A Daybook rule editor would be new work; correction retrieval already exists.

Gemini, Jev, pgvector, Mastra, Wren and managed document search are not integrated. No provider spend or purchase occurred during this research.

Make the next action obvious.

Use everyday tasks as the entrance. Reveal accounting detail when it helps review or understanding.



Record

A short form or sentence; review the full lines before posting. Manual entry stays available.

Review

Drafts and alerts lead to evidence. History and schedules stay accessible. Explain reversals clearly.

Understand

Reports show period and currency. Ask about the books. Account management remains in Settings.

Usability check: a new user can record, confirm and find a payment, then explain its effect without learning the whole accounting vocabulary.

Finish the baseline. Then test one change.

Estimates for one or two days. Docker access and live-provider failures can change the schedule.

0–0.5h	**Preserve the baseline**	Version control, fixture and provider-call limits.
0.5–3h	**Add Gemini adapter**	Proposal, clarification and query paths can run.
3–5h	**Live evaluation**	Check complete entries and evidence; record failures.
5–7h	**Simplify the UI**	Record, review, find and understand a payment.
7–9h	**Docker verification**	Healthy startup and persistence after restart.
9–10h	**Regression checks**	Accounting and affected browser workflows pass.
10–11h	**Phone + key fixes**	Check keyboard, forms and drawers on a device.
11–12h	**Rehearse and package**	Demo, evidence, README and fresh archive.

Hours 12–16: finish core verification first. If it is complete, compare one retrieval change. Framework migration is not a prerequisite.

PROPOSED EXPERIMENT / RESULTS PENDING

A / baseline

Gemini + current text retrieval.

B / challenger

Same setup; add semantic retrieval only.

Same cases, model, ledger, dates and checks.
40 cases: 20 development + 20 held out.

Repeat held-out cases three times per candidate within explicit call/token limits. Keep the held-out set separate from tuning.

Exact task success	Not run
Clarification and evidence	Not run
Latency and cost	Not run

Acceptance requirement: no unconfirmed postings, invalid accepted money or cross-organization evidence observed in the proposed suite. Small tests do not prove zero production risk.

Design a path to growth. Measure the need.

A million registered users does not specify active usage, peak posting rates, report size or model cost.

Today / assignment

One modular API process, PostgreSQL, exact accounting, bounded jobs and shared posting rules.

Before production

Authentication, permissions, backups and restore drills, monitoring, rate limits and benchmarks.

When justified

Separate workers, API replicas behind a load balancer, controlled connections and report summaries.

Accuracy has layers. Database checks protect structure; tests and review protect meaning; source records support explanations.

No live provider accuracy, million-user capacity, failover or physical-device success is claimed by this report.

Follow the evidence

Project records. CHECKPOINT.md, docs/ARCHITECTURE.md, docs/PLAN.md and docs/VERIFICATION.md in the submission.

Spiich. Product description and engineering role. Hiring relevance is an inference.

Midday. Ramp transition; tool selection and matching code.

Actual. Rules, local architecture and currency limitations.

Gemini. Function calling and structured-output limits. Application validation remains necessary.

Optional components. Mastra, Jev, pgvector and Wren. Candidates, not adopted dependencies.

Research checked 29 October 2026. Upstream products were not run or audited in full. This PDF is a native vector companion to the interactive HTML report.